

Mykyta Knysh

CONTACT

Email: nknysh@gmail.com

LinkedIn: www.linkedin.com/in/mykytaknysh

Resume and portfolio online: <https://nknysh.github.io/cv/>

PORTFOLIO

QA Lead – Introduced the first QA function and gave the team confidence in every release

CHALLENGE

The client's company had no formal quality assurance and no QA role. Requirements and defects were not tracked in any consistent way, and nobody could say with confidence whether the software was ready before a release. Every release carried unknown risk, with no evidence to support or challenge the go decision.

MY SOLUTION

- Introduced the QA function to the organization and built and led the team of manual and automated testers (up to 10 people), coordinating QA efforts across it, since there was no existing QA practice to build on.
- Put in place tracking for requirements and defects in Jira, with Zephyr for test management, so that what was expected, what was found, and what was fixed became visible to the team.
- Tested hands-on at every level that mattered before a release: the web and iOS user interfaces, APIs, databases, patches, logs, and third-party service integrations, plus performance testing.
- Added test automation for UI and API checks and integrated it into CI/CD with TeamCity, so release confidence no longer depended on manual effort alone.
- Result: releases moved from unverified to backed by tracked requirements, tracked defects, and test evidence, with a QA team of up to 10 people in place.

TECH STACK

Jira, Zephyr, TeamCity, Nightwatch.js, Cypress, JavaScript, JMeter, Postman

Lead Test Automation Engineer — Built the test strategy, test automation framework and CI pipeline from scratch for a new product and team

■ CHALLENGE

The product and the team were both new, and I joined as the QA lead with no testing strategy or approach to build on. It was unclear what to test, at which level, and how much to automate, so there was no basis for deciding where testing effort should go.

■ MY SOLUTION

- Communicated closely with the product owner and the development lead to clarify requirements and the product architecture, and used that understanding as the basis for the test strategy.
- Defined the test strategy and a test automation roadmap based on product risks, business priorities, technical feasibility, and ROI, so automation effort went first to where it paid off most.
- Set up test automation and CI from scratch and built the suites across layers: both UI and API tests with Playwright, with the API tests covering WebSocket interfaces.
- Interviewed test automation engineer candidates to build the team, then mentored the engineers and built a culture of learning and improvement in the team.
- Result: the team went from no testing approach to a risk-based roadmap and automated UI and API tests running in CI with test reporting, growing into a combined suite of more than 3,600 UI and API tests.

■ TECH STACK

TypeScript, Playwright, WebSocket, Docker, GitLab CI, Postman, Jira, ESLint, SonarQube

Test Team Lead – Unified QA practices across four Scrum teams and cut escaped defects by 70-85%

■ CHALLENGE

Testers in four Scrum teams of a large software product each worked in their own way, with no shared test process, techniques, or quality reporting, so quality could not be compared or steered across teams. Test design was also heavily scripted: testers followed prepared cases that confirmed documented behavior, so edge-case bugs and defects in lightly documented areas, where no script existed, escaped to production.

■ MY SOLUTION

- Led a group of 10 testers embedded across the four teams and owned one shared test process, checking that it was followed or improved, instead of four separate approaches.
- Introduced test design techniques that do not depend on complete documentation: ACC modeling and pairwise testing to cover feature and parameter combinations systematically, and structured exploratory testing with deliberate attempts to break the software in under-documented areas.
- Used statistics and metrics to spot problems across teams and track their resolution, and ran root cause analysis (RCA) on escaped defects to find where process or test design let them through, so changes were driven by evidence, not opinion.
- Mentored and coached the testers on the new techniques and managed the group, so the practices spread through people, not just a document.
- Result: the four teams now shared one process, one set of techniques, and comparable quality data. The number of escaped defects dropped by 70-85%.

■ TECH STACK

MS TFS, MS Test and Lab Manager, MS Hyper-V, Excel pivot tables, MS SharePoint, MS Office

QA Lead — Grew test automation from 50 to 2,200 tests on an up-to-date framework owned by the test team

■ CHALLENGE

The product's test automation framework was outdated and covered very little of the application. The automated tests had been written by software developers without a systematic approach, so the suite was hard to extend and coverage stayed low. Regression confidence depended on a small set of checks that nobody on the testing side owned.

■ MY SOLUTION

- Built a proof of concept to show that a modern framework would work for this product before committing the team to it, then introduced Cypress as the updated test automation framework.
- Set the structure around industry best practices: the page object pattern, fixtures, and reusable Cypress procedures, so new tests reuse existing building blocks instead of starting from scratch.
- Moved ownership of test automation to the testing team, so coverage was planned and grown systematically instead of ad hoc.
- Result: grew the number of automated tests from 50 to 2,200, with coverage now planned and owned by the testing team.

■ TECH STACK

Cypress, JavaScript

Lead Test Automation Engineer — Covered a new reusable library of interactive UI elements with 900+ visual tests and functional UI tests

■ CHALLENGE

A new reusable library of visual, interactive elements was being built to serve as a shared UI foundation. A defect in one shared element would appear in every product that used it, and it could be either a broken interaction or a visual change that no functional check would notice. The library changed continuously while it was being built, so manual checking of how every element looked and behaved could not keep up.

■ MY SOLUTION

- Split the testing into two complementary layers and defined what each one had to prove: functional UI tests for how elements behave, and visual testing for how they look.
- Automated functional UI tests with Playwright to check element behavior and interaction, so a change to one element did not silently break how it worked.
- Added visual testing, also with Playwright, based on a snapshot approach: each element's rendering is compared against an approved snapshot to catch unintended changes in how elements render, which functional assertions do not detect. The trade-off is that every intended design change also fails the comparison, so its snapshots must be reviewed and updated.
- Integrated both suites into CI so every change to the library was checked automatically, with test reporting.
- Result: created more than 900 visual tests, run in CI alongside the functional tests, so unintended behavior and appearance changes were caught before products consumed the library.

■ TECH STACK

TypeScript, Playwright, React, GitLab CI, Jira, ESLint